ARDrums: Air Drums System

Günak Yüzak 2048950, Robert Li 1983696

June 2026

1 Introduction

Drums are a key percussion instrument that brings rhythm to music. Loved by many, and sometimes the key parts of the songs, people love to act as if they were playing it in the air.

Made of many percussion parts, drums are often large and hard to bring to other places. In addition, traditional acoustic drums are very loud and can disturb the people around. The popular proposed solution of electronic drums is still loud enough to bother the close surroundings.

That is why we propose a solution that allows users to play drums without any equipment. The proposed solution is an equipment-free, computer-vision-based virtual drum interface that allows users to play drums using only a standard webcam and computer. The system maps 2D output of the webcam into a 3D coordinate system, allowing for tracking of the user's movements and estimating the relative depth (z) of the user's limbs, thus enabling the mapping of air strikes to specific virtual drum parts. The UI tracks the locations of drums for the user's visibility, and the POV view lets the user see the extension of the limbs in the z-coordinate.

2 Related Work

2.1 Airdrum Sticks

Airdrum sticks [2] are special, commercially available sticks with sensors inside them that can detect hits without any rebound from the surface. They are cheaper than drums, on par with beginner drum pricing, and are quite accurate.

While commercial solutions like this provide high accuracy and a satisfying tactile experience, they inherently limit accessibility since they require the purchase of specialised, proprietary hardware. Furthermore, this system doesn't provide visual feedback to help the user map a virtual drum kit in space.

2.2 A2D:Anywhere Anytime Drumming

In [1], the authors proposed a no-drums approach that uses two drumsticks and a webcam. The system is built upon detecting the drumsticks' location in the frame using the YOLOv5 model that was fine-tuned on a custom dataset consisting of images of air and surface drumming. Additionally, the system utilizes a second-order kinematic Kalman filter to track drumsticks, as the YOLOv5 model has been shown to be somewhat inaccurate.

The proposed solution presents the drum set as a static box within the frame without utilizing the notion of depth and presents an unrealistic model of the drum set. To detect the hit, they utilize a FIFO queue for each stick and store N=4 frames, and determine the hit using these frames.

Authors of A2D provide a Computer Vision approach to the problem. However, the proposed solution is limited by the reliance on the 2D perspective. By representing drums as flat 2D boxes on the screen, the system fails to provide the user with a more realistic experience of playing on a real drum set.

2.3 An Efficient Real-Time Air Drumming Approach Using MediaPipe Hand Gesture Model

In [8], the authors used the MediaPipe hand landmarks model and mapped different hand gestures to different drum parts. The hand gesture makes the sound of that drum part. While the work demonstrated the ability of the model to recognise different hand gestures, the proposed system suffers from the same limitations as [1], since the system fundamentally ignores the actual biomechanics of drumming.

3 Visual Estimation Problems

3.1 Pose Estimation

Pose estimation is the identification and tracking of a person's body. This task presents a significant computational challenge that is nowadays addressed through Deep Learning frameworks such as Ultralytics' YOLO [7] or Google's MediaPipe [6] architectures. These models find a certain number of key points in the body, for example, shoulders, elbows, hips, and so on, and give spatial coordinates for each.

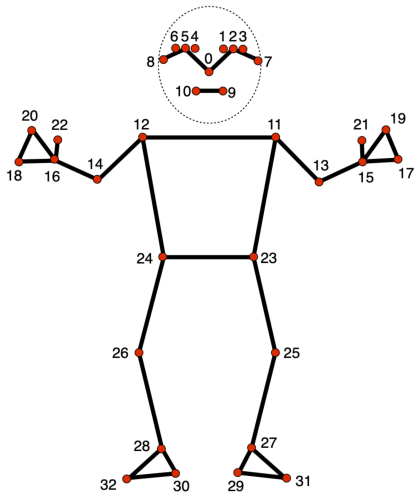

Figure 1: MediaPipe's pose model's output landmarks.

3.2 Hand Estimation

Hand estimation is similar to pose estimation, specifically trained on hands. Many points on hand are tracked for accurate estimation of the hand and fingers' pose.

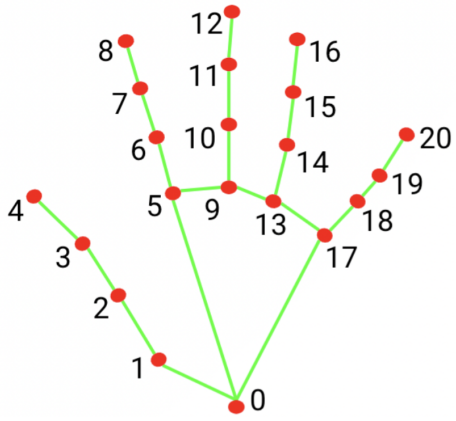

Figure 2: MediaPipe's hand model's output landmarks.

3.3 Depth Estimation

Depth estimation is the task of predicting the depth of objects in the image. The task can be categorized into two subtasks: absolute (or metric) depth estimation and relative depth estimation. Absolute depth estimation aims to predict the absolute distance between the object and the camera, while relative depth estimation aims to predict the depth order of objects or points in a scene without providing precise measurements [4].

In the context of our proposed system, depth estimation is critical for accurate hit detection. At the operating distance of 1-2 meters, the virtual drum set will occupy a small fraction of the frame; thus, individual drum components will appear clustered. As a result, while playing on virtual drums, the user's arm trajectory will often intersect multiple drum parts in the 2D image plane. Meaning that simple 2D collision detection of the arm with drum parts is insufficient. The system requires a depth estimation.

Thankfully, it is possible to estimate depth using the limbs' coordinates, lengths, directions, and angles. In addition, MediaPipe gives estimated z-coordinates. However, the prediction of the MediaPipe model can be inaccurate. To quantify the reliability of the model, we performed empirical validation comparing the model's depth estimation accuracy against known, ground-truth values. The results of the evaluation revealed that the model's accuracy is highly dependent on the position of the hands relative to each other and to the camera. Specifically, the analysis showed a consistent error margin of approximately 3 cm, and in the difficult poses approximately 10 cm.

3.4 Hit Detection

For our project, hit detection is understanding the limbs' movement and determining when there is a hit. For this, we use speed calculation and state logic to detect hits as accurately as possible without detecting as few false positives as possible.

4 Our Method

4.1 Image Preprocessing

Given the unpredictable environment in which the user will use the system (uneven illumination, dark/bright room, source of light being right above or behind the user), as well as limitations that consumer-grade home webcams impose (blurry images during fast movements, automatic exposure adjustment), the system might suffer greatly without preprocessing frames from the web camera.

To mitigate those constraints and improve the further feature extraction done by the Pose Estimation model, we implement a simple preprocessing pipeline that processes each frame through the following sequence:

1. **RGB to CIELAB conversion:** input frames from the web camera are taken in the non-linear RGB color space. Performing any kind of normalisation on such an image will result in severe distortion of the color palette. For this reason, the system converts RGB color representation into a perceptually uniform CIELAB [3] color representation, which allows for the separation of the pure lightness channel (L^*) from the color channels (a^*, b^*). The color data remains untouched during preprocessing, while the following steps are performed on the lightness channel.

2. **Histogram Equalization (CLAHE):** to balance uneven room illumination without globally saturating the image, the system applies Contrast Limited Adaptive Histogram Equalization (CLAHE) [9]. The algorithm partitions the image into patches of 8x8 pixels and performs Histogram Equalization for each patch independently, allowing to amplify details in darker regions while preventing amplification of already bright areas. Due to the fact that local contrast optimisation inherently amplifies noise in the frame, the algorithm also applies clipping to the original histogram.

3. **Edge Detection:** To enhance further feature extraction, the system performs edge detection using a Laplacian edge detector, a second-order differential operator that looks for changes in the slope of change of brightness. Due to the high sensitivity of the Laplacian operator to noise, the system first performs Gaussian blur, computes the Laplacian, and then subtracts the Laplacian from the output of the Histogram Equalization step.

   As the final step, the system combines the preprocessed L channel with the original color channels and converts the image back to the RGB color scheme.

4.2 Initialization

The application begins with a 3-second calibration to allow the user to get into position. By the end of the 3-second countdown, the system captures a few frames and passes them to the Mediapipe model to estimate the coordinates of the shoulders using both Screen and World landmarks. The system averages those coordinates to eliminate potential jitter of Me-

diapipe's prediction. Crucially, the system only accepts frames where both shoulders are visible. The system then calculates the distance between shoulder points, both in Real-World and Screen dimensions. Using these two measurements, it is now possible to calculate baseline metrics and apply augmented reality scaling of the drum set:

- **Metric-to-Pixel Scale:** To ensure that virtual drums are rendered at a realistic scale, the system calculates a conversion ratio (pixels per meter) used to draw the drums at a realistic size relative to the user's body.

$$r = \frac{w_p}{w_m} \quad \text{[pixels/meter]}$$

- **Camera Distance Estimation:** To estimate exactly how far the user is standing from the camera, the system uses a standard pinhole camera model calculation:

$$D = f \cdot r$$

where f denotes the camera focal length (in pixels), w_m represents the physical shoulder width (meters), and w_p represents the measured shoulder width in the image plane (pixels).

- **Focal Length:** Often, it isn't possible to get hardware specifications like the focal length of the web camera. So instead, the system employs a standard pinhole model and calculates focal length empirically. The calculation of focal length is performed under the assumption that the Field-Of-View (FOV) of the web camera of the user is $\approx 60°$ (typical for standard consumer web cameras). Since we know the exact width of the frame and use an assumption about the FOV of the webcam, we can calculate the distance as follows:

$$\mathrm{f} = \frac{\frac{\text{Frame Width}}{2}}{\tan\left(\frac{\text{FOV}}{2}\right)}$$

Using the calculated distance from the camera to the user and the focal length, the system can scale the drum set through perspective projection. It's important to note that the calculation of the distance from the user to the camera can be avoided completely since the system calculates the real-world shoulder width of the user and can potentially scale the drum set according to this measurement. However, the distance to the camera can be an important metric, since the Mediapipe model performs best when the user is at a certain distance from the camera. The calculation of the distance can allow the system to "guide" the user to the predefined "sweet spot" (about 1.5-2 meters from the camera).

The app launches threads for the camera, for the MediaPipe detection, and for wrist and foot processes. Multithreading helps with latency, as different operations do not cause latency in later operations. Without threading, if the left wrist works before the right wrist, then the right wrist will have a slight but audible delay.

If the user prefers, there is a version that adds virtual, drawn, and calculated sticks to the user's hands.

4.3 Pose Tracking

We use MediaPipe's heavy model to find the landmarks, which are the important points, on the user's body. For each landmark, MediaPipe gives two (x,y,z) values, which are screen landmarks and world landmarks. We tried YOLO, but we saw from the drawn skeleton model that it was less accurate.

Screen landmarks are absolute pixel coordinates in normalized image coordinates. For the z-coordinate, there is no pixel coordinate, but a relative distance value. The x-y coordinates are normalized by the screen width and height.

World landmarks are distances in meters, where the origin is the middle of the two hip points, right and left. In this case, z is also the estimated distance in meters, which is more accurate than the screen landmark version, but we still observed many inaccuracies coming from z.

We take the left and right wrist landmarks and right foot landmarks and pass them to hit detection.

We also use a Kalman filter to produce a smooth, consistent 3-D wrist estimate (kx, ky, kz) from noisy MediaPipe outputs so the hit logic uses reliable coordinates and speeds.

4.4 Hand Tracking

We tried MediaPipe's hand tracking model, but unfortunately, for a task such as drumming tracking, the hands are hard to track. The hands are always closed, while the model is most accurate when the hands are open. The hands are always moving; thus, they are always blurry; therefore, the tracking fails.

These two issues caused the drawn skeleton to disappear constantly and reappear again, indicating that the model can detect but can not follow the hands. Therefore, we only relied on pose tracking and discarded hands.

If the hand tracking worked, overall hit detection quality could be improved as calculations drawn from the finger locations can be added on top of already existing wrist calculations, thus classifying hits better. Additionally, with finger locations, finger stroke techniques, which are widely used, could have been implemented.

4.5 Depth Estimation

Initial evaluation of the prototype of the system revealed that Mediapipe's internal z-coordinate estimation lacked the precision required for our system. As was mentioned earlier, further evaluations showed an error margin of approximately 8 cm.

Given this limitation, we determined that an external depth estimation approach was necessary. As a result, the depth estimation method is inspired by the work [5] on estimating 3D Human Body Postures from a single view. By treating the upper and lower arms as segments of constant length, we can reconstruct their 3D positions from the projected 2D lengths in the camera frame.

To establish the physical lengths of the lower and upper arms, during the calibration, the system takes measurements of the length of the upper and lower arms of the user and saves them for later use as fixed constants.

The z are calculated in three steps:

- Projected lengths of upper and lower arm on camera space are found by taking the distance between the elbow and shoulder, and the wrist and elbow in the image plane.

  $$\sqrt{(elbow_x - shoulder_x)^2 + (elbow_y - shoulder_y)^2}$$

  These pixel-based distances are converted into meters using a pre-calculated pixel-to-meter scaling factor derived during the calibration phase.

- Given the known physical length of each limb segment (L_{real}) and its projected length ($L_{\text{projected}}$), we estimate the depth offset (d) using the Pythagorean theorem:

  $$d = \sqrt{\max(0, L_{\text{real}}^2 - L_{\text{projected}}^2)}$$

- The z-coordinate of elbow,elbow_z, is $-d_{\text{upper}}$, while the z-coordinate of wrist is elbow_z $+d_{\text{lower}}$ if the wrist is behind elbow, or elbow_z $-d_{\text{lower}}$ if the wrist is in front of the elbow.

These calculations give negative z-coordinates as the landmarks approach the camera; the hip and shoulder z-coordinates are given 0, in line with MediaPipe values.

To understand whether the wrist is behind or in front of the elbow, MediaPipe z-coordinates are used.

In addition, we detect when the user adopts a typical drumming posture and apply a small, smoothly time-varying forward offset to the wrist z-coordinates produced by the pose estimator. The offset forward (toward the camera) compensates for conservative depth estimates and improves intersection tests for drum hit detection.

4.6 Hit Detection

For hit detection, we need to be able to tell when the joint's movement could indicate a hit. From all possible wrist and arm movements, what is more important for drums is downwards and upwards strokes, where the hit is carried through the downwards motion and returned to the original position through the upwards motion to get ready for the next hit. With that, the transition points between the two also carry importance to tell when the hit has been handed and when the next hit is ready.

To define the two strokes, we define states UP and DOWN, which represent when the joint's movement is upwards and downwards. For state transition, we need to check for speed at each frame and see which direction it is and if it is above a certain movement threshold. Any movement with a speed lower than the threshold is not considered.

The speed is calculated by the change in the wrist's screen coordinates between the previous frame and the current frame. It is then normalized by the calculated shoulder width.

$$\texttt{raw\_norm\_speed} = \sqrt{\left(\frac{x_t - x_{t-1}}{s}\right)^2 + \left(\frac{y_t - y_{t-1}}{s}\right)^2}$$

The change in y is considered separately, as downward motion, to determine the direction the joint is heading.

For additional accuracy and support, we implemented an optical flow system that looks around the wrist and also calculates the change using the flow vectors. The coordinate speed and the flow speed are then averaged for a joint representation.

Now that we can tell when the movement could carry a hit with this state change logic and speed calculation:

- If the speed is above the hit threshold,
- And the state is down,
- And if enough time has passed since the previous hit:

Then we have a valid hit, and we check for the drum part.

On a successful drum part hit, the drum part plays its sound. After the hit, we put the wrist that hit on a cooldown, where the hits will not be rejected until the cooldown is over. This blocks multiple hits in a single stroke and ensures fewer false positives and better performance.

If there wasn't a drum part and it was empty, then there is no hit, and we do not set the cooldown.

We also implemented a frame count for a state change. The state only changes if the speed is above the threshold for a certain number of frames. This reduces false hits caused by jittering. Unfortunately, it also makes the model not consider hits above a certain RPM, as there are not enough frames before the wrist goes up to turn to state down.

To allow horizontal hits, when moving between two parts only in x coordinates, the hit logic also checks if the horizontal motion is larger than a threshold. This way, going between toms and cymbals works.

4.7 Drum Part Checking

After hit detection, we need to understand if it hit a drum part, and which part it was that got hit.

Each drum part is defined as an ellipse, with a center coordinate in (x, y, z) and radii for each of the coordinates (r_x, r_y, r_z).

Then, using the location of the wrist (w_x, w_y, w_z), we first check if w_z is in the boundaries of the z-coordinates, that is, $w_z \in (z - r_z,\ z + r_z)$.

This first check is used to determine the correct drum part in the z location, ensuring overlapping parts are separated. Then we check the ellipse equation in the x-y coordinate for every part that passed the z check.

$$\frac{(w_x - x)^2}{r_x^2} + \frac{(w_y - y)^2}{r_y^2} \leq 1$$

If the equation holds, then the wrist is on that part, and the sound of that part is played.

Each drum part also has an idle color for display purposes and a sound file path, which is played when the wrist is in the ellipse.

A special case is the bass drum, where there is no need to see if it intersects the ellipse or not, as it is a relatively simple movement, just bringing the leg up and down. Feet do not have the same freedom and expression as wrists; simple hit checking is enough. The center coordinates and radii are for drawing purposes.

In total, we have implemented hi-hat, crash, ride cymbals; snare, left tom, right tom, and bass.

4.8 UI Elements

To show different useful information for both debugging and user experience, we draw the information on the screen, toggled by certain keyboard keys. These include drums, drum names, coordinates, flow, POV, and many more.

We draw drums on the main screen based on their center (x,y) coordinates and radii (r_x, r_y). Hitting a drum is simply passing your wrist in the drawn circle on the screen. Unfortunately no real way to show the z distance by drawing drums alone, but we made the placements intuitive and comfortable.

POV view is a screen to the side. It shows a drawing of hands and arms, and makes it possible to visualize z. Here, the drum parts are drawn similarly to their real-life shapes, instead of ellipses on the screen. The hands and arms move according to the user's input, allowing them to see where their hands are relatively, and how close they are to the drum parts they want to hit.

To calculate the POV vision, we first put the drum part's coordinates to the center of the camera. Then we apply rotation changes to make it look like POV. Finally we scale the parts based on their distance (z coords). To render the parts, we use painter's algorithm, which is simply drawing the farthest first, since if we drew the closest parts first they would be later obsured by what they were supposed to be in front of.

5 Results

5.1 Hit Accuracy

We conducted tests on the hi-hat at 60 bpm and 90 bpm. We considered how many hits were done in 15 seconds, which is 15 for 60 bpm and 22 for 90 bpm.

Below is the average hit accuracy for all samples on the hi-hat part.

Subjects	60bpm	90bpm
Sub1	0.9048	0.7792
Sub2	0.8667	0.9091

5.2 Depth Error

With a measurement tape, we measured how accurate the landmark z-coordinates are, compared to the real-life metric distance. World landmarks are already in meters, and make it easier for us to see any inaccuracies.

We mostly saw very accurate calculations and insignificant error, but there are certain cases that made the error 10+ centimeters. One such case is when the wrists are close together. In a pose where it said right wrist was 30ish centimeters away while the left wrist was on the hip, after extending left the same way, it said 20ish for right. The movement of the left should not bring the right lower; in fact, the real arm distance to the body was 30ish the entire time.

5.3 Latency

We calculated the latency between when the hit was detected and when the hit part plays its sound. The observed latency was in the range of [0.02, 0.11] ms.

We note, however, that this latency may change depending on the hardware. It can be due to multiple reasons, such as slower CPU and GPU, or having an external camera instead of the webcam on laptops. We observed these in our testing, and we noticed the delay present when the camera was external. Even in this case, however, the delay is mostly unnoticeable and needs comparison to notice its existence.

6 Limitations

Many limitations in this project impacted the final accuracy. The first one is the camera frames per second. The only cameras available for us were 30fps, while 60fps cameras would have a great effect on accuracy and timing. Important data for state change and hit timing can be at a time that could not be captured as a frame.

We tried interpolating the frames to obtain 60fps per second artificially, but it did not help, and it slowed down the performance massively on lower-end computers.

Another limitation is depth estimation, whose inaccuracy had a massive impact on the performance. We improved this by ourselves as much as we could and obtained a working result, but with better estimations, the accuracy can be greater.

Finally, as mentioned above, we tried to use hand landmarks instead of wrists, but available hand landmarks were not suitable for drumming, as often, the hands were closed, and fingers were behind. A possible solution is fine-tuning the model on closed hands or a fully trained model specialized for drum data and performance. Additionally, a higher-quality camera can help with the blurriness resulting from fast movements.

References

[1] A2D: Anywhere Anytime Drumming — arxiv.org. https://arxiv.org/abs/2304.03289. [Accessed 29-05-2026].

[2] Aerodrums — Air Drums & Virtual Electronic Drum kit — aerodrums.com. https://aerodrums.com/. [Accessed 29-05-2026].

[3] Colorimetry — Part 4: CIE 1976 L*a*b* colour space — CIE — cie.co.at. https://cie.co.at/publications/colorimetry-part-4-cie-1976-lab-colour-space-1. [Accessed 02-06-2026].

[4] What is Depth Estimation? - Hugging Face — huggingface.co. https://huggingface.co/tasks/depth-estimation. [Accessed 31-05-2026].

[5] Hsi-Jian Lee and Zen Chen. Determination of 3D human body postures from a single view. *Comput. Vis. Graph. Image Process.*, 30(2):148–168, May 1985.

[6] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg, and Matthias Grundmann. Mediapipe: A framework for building perception pipelines, 2019.

[7] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[8] Pabasara Surasinghe, Pasan Herath, and Kokul Thanikasalam. An efficient real-time air drumming approach using mediapipe hand gesture model. In *2023 5th International Conference on Advancements in Computing (ICAC)*, pages 215–220, 2023.

[9] Karel Zuiderveld. Contrast limited adaptive histogram equalization. In *Graphics Gems*, pages 474–485. Elsevier, 1994.